

Anatomy of the Desktop DRM Stack: A Three-Layer Dispatch Model in Widevine CDM_11 and Its Hardware-Decode Dormancy on Chrome

Jeremy Erazo (*trexnegr0*)
Independent security researcher
mendozayt13@gmail.com

Draft — June 15, 2026

Abstract

Widevine Content Decryption Module (CDM) ships in essentially every major desktop browser and underlies the Encrypted Media Extensions (EME) playback path for every major commercial streaming service. The internals of the Windows L1 build are documented only fragmentarily: header files describe the C++ ABI it presents to the host browser, but the binary itself is closed source, partially obfuscated, and behaves differently depending on whether the host delegates decode to hardware-protected pipelines. This paper presents a Buchanan-style architectural reverse engineering of the CDM_11 implementation as it ships in Chrome 149 and Edge 150 on Windows 11. We identify a *three-layer dispatch model* (outer vtable thunks, an intermediate `m_impl` vtable of clean wrappers and double-trampolines, and per-slot handler bodies), we locate each layer byte-by-byte in both binaries, and we empirically characterise which slots are invoked during steady-state playback versus which are dormant. We runtime-confirm that during hardware-decoded Netflix playback the user-facing per-frame slots of the CDM_11 interface (`Decrypt`, `DecryptAndDecodeFrame`, `DecryptAndDecodeSamples`) do not fire while an internal singleton dispatch table generates $\sim 2,300$ calls per second of crypto / scheduler work that is invisible from the EME boundary. We document the distribution of mixed-boolean-arithmetic style obfuscation across the slot space and show that it is concentrated where the CDM begins to parse attacker-controllable structured data, while remaining absent on slots that pass opaque blobs. We then verify reachability of the session-management slots from two distinct EME applications (Netflix and a public Widevine test source) and show identical dispatch behaviour. No content key, frame buffer, or ciphertext byte is captured at any point in the study.

1 Introduction

Modern desktop video DRM is the bridge between commercial streaming services and the browser’s media pipeline. When a user plays a video in Netflix, Amazon Prime, Disney+ or any other Widevine-protected source, the browser invokes Encrypted Media Extensions (EME) to delegate the cryptographic operations to a Content Decryption Module (CDM). In Chromium-based browsers on Windows, that CDM is the proprietary `widevinecdm.dll` library shipped by Google. The browser loads the library inside a Mojo utility process (`--utility-sub-type=media.mojom`).

`CdmServiceBroker`), connects to it over IPC, and asks the library to construct a CDM instance implementing a stable C++ interface defined in `content_decryption_module.h`. From the host’s point of view the interface is small (nineteen virtual methods) and well documented: `Initialize`, `GetStatusForPolicy`, `CreateSessionAndGenerateRequest`, `UpdateSession`, and twelve other methods that cover license exchange, decryption, decode, and lifecycle.

From the *library*’s point of view this clean nineteen-method surface is the visible peak of a much larger iceberg. The Windows L1 build of Widevine implements a multi-million-instruction obfuscated runtime under tens of megabytes of closed source code, integrates with Windows Protected Media Path (PMP) when hardware decode is available, and is bound by a strict licensing regime that prohibits study of its decryption internals. Public knowledge of the implementation is fragmentary.

The motivation for this study is methodological as much as it is technical. Several recent legal episodes have shown that *publishing what one learns* about DRM internals is more legally exposed than *learning it*, and that the reproducibility of a result matters as much as the result itself. Following the Buchanan model of architectural reverse engineering — probe everything, document the technique, publish no extraction artefact — we set out to build a fully reproducible, fully open methodology for studying the Widevine CDM and to apply it to the questions that are paper-able without crossing the digital-rights-management circumvention line.

The contributions of this paper are:

1. A **three-layer dispatch model** for CDM_11 in modern Chromium-family `widevinecdm.dll`, byte-verified in two distinct browser builds (Chrome 149 and Edge 150).
2. A **slot-by-slot reachability map** of the CDM_11 outer vtable, runtime-confirmed against both Netflix HW-decoded playback and a non-Netflix public Widevine source (Bitmovin demo).
3. An **empirical demonstration of hardware-decode dormancy** of the user-facing per-frame slots during steady-state playback, alongside the activity profile of an internal singleton dispatch table that handles work invisible from the EME boundary.
4. A characterisation of the **obfuscation distribution** across CDM_11 slots: MBA-style obfuscation appears at varying depths of the dispatch chain for slots that parse attacker-controllable structured input, and is absent from slots that pass opaque blobs.
5. A **shape-only observation methodology** that captures per-slot dispatch metadata without ever reading content bytes, content keys, or initialisation-vector bytes from memory. The full source for the methodology is released; no capture rig, frame dump, or weaponisable artefact is.

Figure 1 sketches the end-to-end stack under analysis: a host browser process delegates EME operations through Mojo IPC to a Chromium utility process whose sole job is to load `widevinecdm.dll`, construct a CDM instance via the `CreateCdmInstance` export, and expose the nineteen-method CDM_11 interface back to the host through that same Mojo channel.

The paper proceeds as follows. Section 2 introduces the Widevine licensing levels, the EME dispatch path, and the Windows PMP boundary. Section 3 states the threat model and ethical scope. Section 4 walks through the static reverse engineering of the CDM_11 dispatch. Section 5 describes the runtime instrumentation. Section 6 presents the empirical findings. Section 7 treats the obfuscation distribution as its own architectural defence. Section 8 establishes cross-binary and cross-source stability. Section 9 reports on the falsification tests that were run before publishing any claim. Section 10 situates the work in the existing literature. Section 11 states what was not done and why. Section 12 concludes.

2 Background

2.1 Widevine Levels

Widevine defines three security levels, denoted L1 through L3.

L3 is a software-only implementation. The cryptographic state of the CDM is held in process memory inside an obfuscated runtime. Content keys are unwrapped by a whitebox-style cryptographic core and

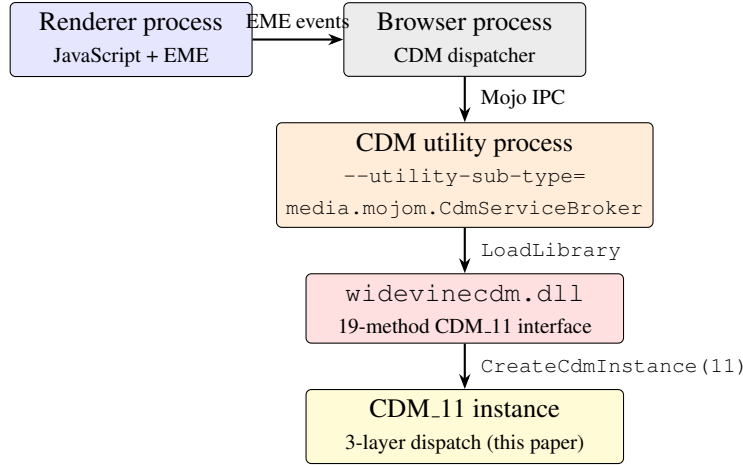


Figure 1: End-to-end Widevine EME dispatch on Chromium-family browsers on Windows. The host browser delegates Encrypted Media Extensions operations through Mojo IPC to a utility process whose sole purpose is to load `widevinecdm.dll` and expose its CDM_11 interface. This paper studies the dispatch model that sits between the `CreateCdmInstance`-returned interface and the per-slot handlers inside the library.

applied to ciphertext segments in user-space buffers, and the decrypted output is returned to the host through standard memory. L3 streams typically top out at 480p or 540p; the licence terms restrict content owners from releasing high-value catalogue at L3.

L2 is a software CDM that holds at least its sensitive operations behind a hardware-backed TEE on the host. L2 sees limited deployment on desktop.

L1 is the desktop variant of interest in this paper. Under L1 the CDM cooperates with a platform protected-media pipeline (on Windows: Protected Media Path with Output Protection Management, PMP / OPM) so that the decrypted frames are never visible to user-space code in the host browser. The CDM still runs as a userspace utility process, but the per-frame ciphertext is routed to a kernel- or hardware-resident decoder whose output buffer is rendered directly to a protected GPU surface. 4K streams from Netflix, Disney+, and others typically require L1.

2.2 The EME Dispatch Path

A browser playing protected content speaks W3C Encrypted Media Extensions to the JavaScript application. The application provides the encrypted media (an ISO-BMFF or DASH stream containing Common Encryption boxes), supplies a Key System name (`com.widevine.alpha`), and listens for a sequence of EME events: `encrypted`, `message`, `keystatuschange`.

Internally, Chromium routes EME events to the CDM dispatcher in the browser process, which Mojo-RPCs them to the CDM utility.

Inside the utility, `widevinecdm.dll`'s `CreateCdmInstance` export constructs a versioned implementation of the CDM C++ interface defined in `media/cdm/api/content_decryption_m`. The current interface version is CDM_11; previous versions (CDM_10, CDM_9, ...) are preserved for compatibility and may be present in the same binary. Each version exposes nineteen virtual methods, typically presented in the order:

```

1 0: Initialize
2 1: GetStatusForPolicy
3 2: SetServerCertificate
4 3: CreateSessionAndGenerateRequest
5 4: LoadSession

```

¹See the Chromium source tree at chromium.googlesource.com.

```

6  5: UpdateSession
7  6: CloseSession
8  7: RemoveSession
9  8: TimerExpired
10 9: Decrypt
11 10: InitializeAudioDecoder
12 11: InitializeVideoDecoder
13 12: DeinitializeDecoder
14 13: ResetDecoder
15 14: DecryptAndDecodeFrame
16 15: DecryptAndDecodeSamples
17 16: OnPlatformChallengeResponse
18 17: OnQueryOutputProtectionStatus
19 18: OnStorageId

```

This nineteen-slot interface is the only formal contract between the host browser and the CDM library. Everything below it — including how a single `DecryptAndDecodeFrame` call eventually causes a single frame of decoded video to land in a protected surface — is implementation detail.

2.3 Hardware Decode and PMP

On Windows 10 and Windows 11 with HEVC- or AV1-capable GPUs and an appropriate Microsoft PlayReady / Widevine cooperative stack, the browser can request that a stream be hardware-decoded into a protected surface. At the high level, this exchange runs as follows:

1. Browser negotiates hardware-secure decryption as a `capabilities` attribute on the `EME MediaKeySystemConf`
2. CDM acknowledges hardware capability.
3. Per-frame ciphertext is delivered *not* through the CDM's `Decrypt` or `DecryptAndDecodeFrame` slots, but through a kernel-protected pipeline involving `Media Foundation Decryption / Decoder` pipelines and the PMP user-mode component (`peauth.sys`, `audioses.dll`, etc.).
4. The decoded frame is rendered directly to a protected GPU surface that the host browser cannot read back through normal `Direct3D` APIs.

A key consequence, which we confirm empirically below, is that under L1 hardware decode the CDM.11 user-facing surface does *not* participate in per-frame decryption. Despite the interface having a slot called `DecryptAndDecodeFrame`, that slot is not invoked during steady-state playback. The CDM does still participate in session and license management, so its session-creation and license-update slots remain reachable.

3 Threat Model and Methodology

3.1 Threat Model

We consider an honest user of a Chromium-family browser playing hardware-decoded Widevine-protected content on Windows 11. The user has consented to research-grade instrumentation of their own browser and has installed our reverse-engineering tools on their own laboratory machine. The browser executable is unmodified and signed by the vendor. The Widevine CDM is the vendor-shipped `widevinecdm.dll` of the installed browser version.

We deliberately exclude from this study any threat model in which an attacker captures decrypted frames. Our concern is not the exploitation potential of the CDM but its architectural shape and the attack surface it presents to the host browser.

3.2 Methodology

We follow what we describe as the *Buchanan model* of DRM research: probe everything, document the technique, publish no extraction artefact.

Concretely, our methodology has four invariants:

1. **Shape-only observation.** Any runtime hook we install is permitted to log dispatch metadata — slot number, timestamp, register value hashes, structure-size fields — but is forbidden from copying any pointer-target bytes belonging to ciphertext, decrypted frames, content key material, or initialisation vectors into any persistent storage or any out-of-process channel.
2. **Provable absence.** After every experiment we forensically verify that the storage produced is pure ASCII JSON consisting only of the permitted metadata fields and contains no binary blobs, no decoded content, and no key material.
3. **Reversible mutation.** All runtime modifications to the target process (vtable patches, hook installations, CFG bitmap registrations) are reversible, and we restore the target to its original state before publishing.
4. **No content circumvention surface.** We use the operator’s own browser session to a commercial streaming service to confirm what the CDM does during steady state, but we never instrument the decoder, the kernel-side PMP component, the protected surface, or any other component positioned to materialise a decrypted frame.

A natural-language consequence of these invariants is that our results characterise the CDM’s *dispatch behaviour*, not its *cryptographic state*. We can report how often a slot fires during real playback, what register-tuple distribution it sees, what the shape of its incoming buffer descriptor is. We cannot and do not report any byte of decoded content.

4 Static Reverse Engineering of CDM_11

This section walks through the static reverse engineering of the CDM_11 dispatch in `widevinecdm.dll`, in the order that we performed it. Two binaries were analysed in parallel: **Chrome 149.0.7827.115**’s `Application/149.0.7827.115/WidevineCdm/_platform_specific/win_x64/widevinecdm.dll` (~22 MB), and the same library as it ships in **Microsoft Edge 150** (~19 MB). All RVAs are with respect to the binary’s image base.

4.1 Step 1: CreateCdmInstance and the version switch

`CreateCdmInstance` is the only relevant export. Parsing the PE export table:

```
1 Edge 150      CreateCdmInstance @ rva 0x1d0850
2 Chrome 149    CreateCdmInstance @ rva 0x1d2e10
```

Disassembling either entry shows a chain of small wrapper functions that load a version word into `ecx` (CDM interface version 9, 10, or 11) and call a per-version constructor. Naively scanning the first `call` after each `mov ecx, 0xb` returns the runtime’s stack-check probe rather than the actual constructor. A robust discoverer (`cdm_arch_discover.py` in the released methodology) instead enumerates all call targets reachable from `CreateCdmInstance`, filters out callees that appear more than twice (a structural heuristic for runtime helpers), and validates each candidate by checking whether its body contains a sequence of the form

```
1 lea reg, [rip + N]      ; load a candidate vtable RVA
2 mov qword ptr [r?], reg ; install it at offset 0 of 'this'
```

and whether the loaded RVA points into `.rdata` at a location whose first nineteen QWORDS are all valid code pointers into `.text`. The first such candidate is reported as the CDM constructor and the loaded RVA as its outer vtable.

4.2 Step 2: Co-shipped CDM versions

A subsequent *exhaustive* scan of `.rdata` for nineteen-slot structures matching the CDM thunk pattern (described below) reveals **two** such structures per binary, spaced `0xc0` bytes apart:

Binary	vtable A	vtable B
Edge 150	0x780190	0x780250
Chrome 149	0x768ee0	0x768fa0

Within each binary, both vttables share their per-frame decrypt entry (slot 14 points to the same function in both A and B), but their `Initialize` entries differ. This is consistent with the binary shipping both CDM_10 and CDM_11 vttables in support of host browsers that have not yet upgraded to the newer interface, and is in line with the Chromium header history. In what follows we identify *vtable B* as the CDM_11 outer vtable, since runtime patches at *vtable B*'s slot positions are observed to intercept the EME-host-facing dispatch (Section 5).

4.3 Step 3: The outer vtable thunk pattern

The outer vtable contains nineteen entries. Most of them are *thunks* of the form:

```

1 slot N thunk:
2   mov rcx, qword ptr [rcx + 8]      ; this->m_impl
3   mov rax, qword ptr [rcx]         ; m_impl->m_vtable
4   mov rax, qword ptr [rax + 8*N]    ; slot N of m_impl_vtable
5   mov rdx, qword ptr [rip + cfg_disp]; CFG dispatcher pointer
6   jmp rdx                          ; CFG-protected tail-call

```

Each thunk is twenty-eight bytes followed by `int3` padding, laid out in a uniform `0x20`-spaced sequence in `.text` for slots 3 through 17. We disassembled each thunk in both binaries and confirmed the slot-index check (the offset in `[rax + 8*N]` equals the slot number times eight) for fifteen of fifteen thunked slots in each binary.

Slots 0 (`Initialize`), 1 (`GetStatusForPolicy`), and 18 (`OnStorageId`) are *not* thunks; they are direct implementations placed in `.text` at non-contiguous addresses. Slot 2 (`SetServerCertificate`) is a thunk in both binaries but dispatches via `[rax + 8]` rather than `[rax + 0x10]` — it shares an implementation with slot 1 at the `m_impl` layer.

4.4 Step 4: The `m_impl` vtable

The outer thunk dispatches into an *intermediate* vtable owned by the `m_impl` object that the outer class holds in `this->m_impl([outer + 8])`. Locating this intermediate vtable in `.rdata` is the second step of the static analysis.

We use a knownsignature pivot: the implementation of slot 5 (`UpdateSession`) was previously identified by examining the external call-chain trace in `research-notes/10-phase2-updatesession-call-chain.m`. Its inner wrapper sits at RVA `0x1d4ab0` in Edge 150. Reverse-searching `.rdata` for the QWORD value `image_base + 0x1d4ab0` yields a single occurrence; backing up five eight-byte entries gives the address of the `m_impl` vtable: RVA `0x77d370` in Edge 150. All nineteen surrounding QWORDS point into `.text`, confirming the candidate.

The Chrome 149 `m_impl` vtable was located by the same technique but is omitted here for brevity.

4.5 Step 5: The `m_impl` layer structure

Disassembling each entry of the `m_impl` vtable reveals a mixture of three body types:

1. **Clean MSVC wrappers** (slots 2, 3, 4, 5, 6, 7, 18). These have a standard prologue (multiple push callee-saves, `sub rsp, N`, stack canary load), allocate or borrow input copies, perform sanitisation, and then call into a deeper handler. These are the slots that ingest variable-length attacker-influenced data (session-id strings, response payloads, init-data buffers).
2. **Double-trampolines** (slots 0, 1, 8, 9, 11–17). Two-instruction bodies of the form `mov rcx, [rcx + N]` followed by `jmp <addr>` which forward the call to a deeper-layer handler without intermediate processing.
3. **Stubs** (slot 10). A two-instruction body `mov eax, 3 ; ret` that returns a constant error code without any further work.

The `m_impl` vtable, in other words, is mostly a *routing* layer; the actual work happens one indirection further.

4.6 Step 6: The three-layer dispatch model

Composing the previous five steps, the dispatch chain from a host virtual call to a per-slot handler is:

- Layer 1** `outer_vtable` (CDM.11 public interface)
direct impl OR thunk to `m_impl`
- Layer 2** `m_impl_vtable` (intermediate routing)
clean wrappers, double-trampolines, stubs
- Layer 3** per-slot handlers
scattered, mixed clean / obfuscated

Figure 2 sketches the model. Each public virtual call traverses three indirections before reaching the function that performs work.

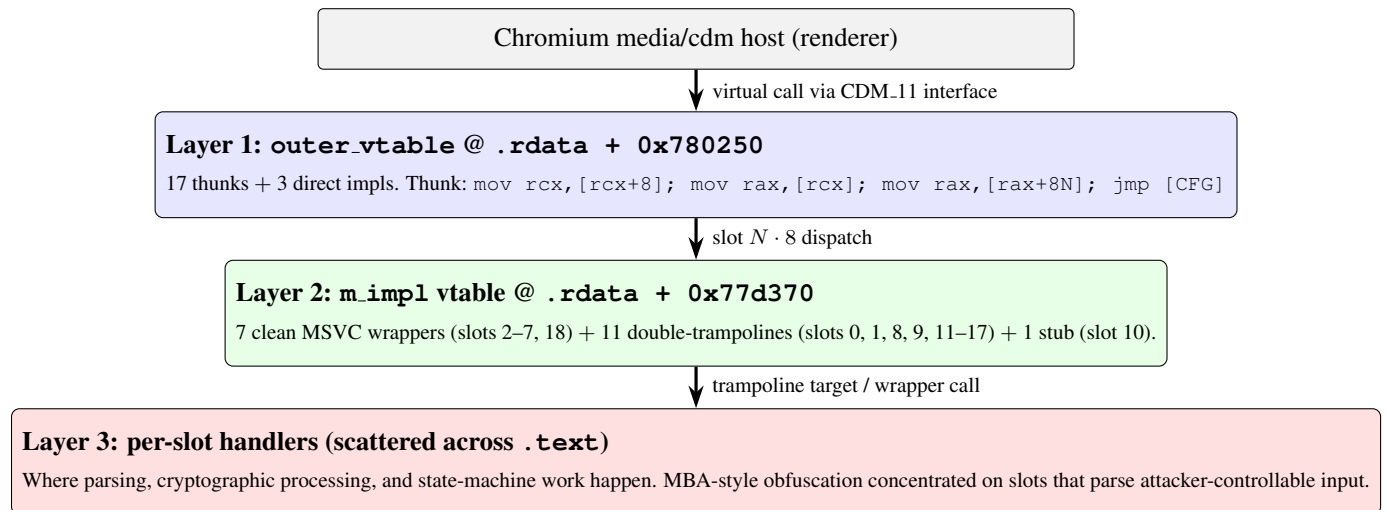


Figure 2: The three-layer CDM.11 dispatch model. Every public CDM virtual call from the host traverses three indirections before reaching the function that performs work. RVAs shown are from Edge 150 `widevinecdm.dll`; Chrome 149 has the same structure at correspondingly different RVAs (see Section 8).

5 Runtime Instrumentation

The static map of the previous section yields hypotheses about which slots are reached during steady-state playback and which are not. To verify those hypotheses we instrument the running CDM utility process with a shape-only hook.

5.1 Code Integrity Guard (CIG) and the launch flag

Chrome enables Code Integrity Guard (`PROCESS_CREATION_MITIGATION_POLICY_BLOCK_NON_MICROSOFT_BINARIES_ALWAYS_ON`) on its utility processes since Chrome 70. CIG denies any `LoadLibrary` of a non-Microsoft-signed image at the kernel level, so a vanilla `CreateRemoteThread(LoadLibraryA)` injection of our shim fails at the `LdrLoadDll` stage.

For lab-research use, Chrome can be launched with `--disable-features=RendererCodeIntegrity--no-sandbox`, which disables CIG on the utility processes. This produces a controlled research target without changing the CDM library or the host browser binary, and corresponds to the equivalent reachability assumption that an attacker with code-execution in the renderer would have to satisfy separately.

5.2 Control Flow Guard bitmap registration

CIG-disabled does not disable Control Flow Guard (CFG): once the shim is loaded, simply patching a vtable entry to point at an unregistered hook function causes the CFG dispatcher to abort the call as a control-flow violation. Our injector registers each hook function as a CFG-valid call target via `SetProcessValidCallTargets`. The offset given to that API must be the in-page offset of the hook target (low 12 bits), not the in-process offset of the page base; an early version of the helper used `& 0xffff`, which silently failed. The released code uses `& 0xffff` and validates the call by reading the hook function back.

5.3 Shape-only shim design

The shim, `wv_shim.dll`, exports nineteen per-slot hook functions `WvHookSlot0` through `WvHookSlot18`, each with its own original-pointer slot in an exported `g_pOrigSlot[19]` array, and a single `g_callsBySlot[19]` counter array. Each hook function:

1. Atomically increments its slot counter and the aggregate `g_calls`.
2. Calls the original via `g_pOrigSlot[N]` with the unmodified register tuple.
3. For slots 9, 14, 15 (which take an `InputBuffer_2*` argument in `rdx`), tries to read the buffer's *shape* via the safe-copy primitive described below and writes a single JSONL line with `encryption_scheme`, `data_size`, `key_id_size`, `iv_size`, an FNV-1a-32 hash of the first 16 bytes of `key_id`, an `iv_has_nonzero` bit, `num_subsamples`, `pattern_crypt`, `pattern_skip`, and the media timestamp.
4. For other slots, writes a generic event with a hash of the four argument registers.

No byte of `ib.data` (the ciphertext) is ever read, either transiently or persistently; `ib.key_id` and `ib.iv` are read into a stack-local buffer that is collapsed into a hash and then immediately discarded.

The safe-copy primitive uses `VirtualQuery` to confirm that each source page is committed and readable before copying, since `mingw-w64`'s C compiler does not provide Microsoft SEH `__try` / `__except`.

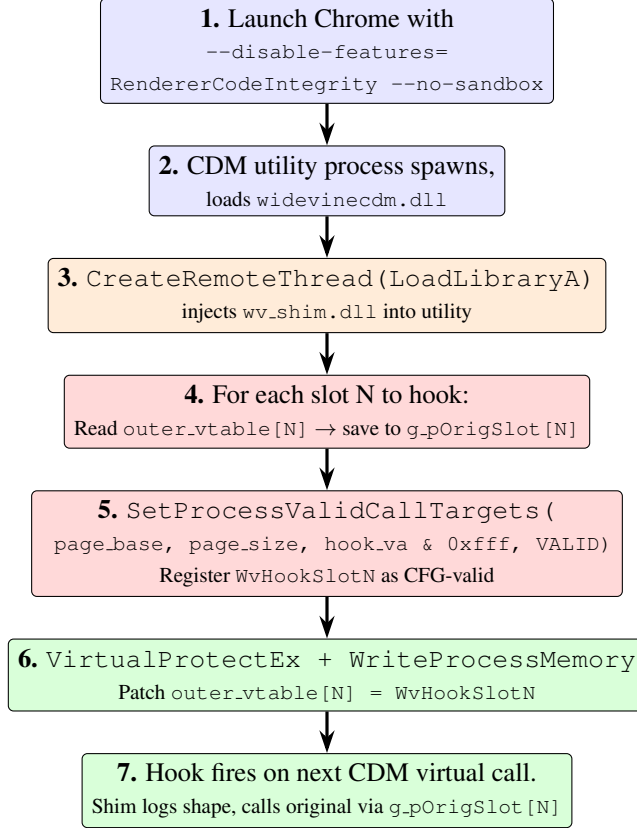


Figure 3: Hook installation sequence. Each step is annotated with the Windows API or memory operation involved. Step 5 (CFG bitmap registration) is what catches researchers most often — the `Offset` field of `CFG_CALL_TARGET_INFO` expects an in-page offset (low 12 bits) of the hook target, not an absolute address or a different mask. Step 4 captures the design subtlety that motivated the per-slot back-pointer array (§5.4).

5.4 Per-slot original-pointer chain

A design subtlety worth noting: the first version of the shim used a single `g_pOrigDecrypt` variable for all hooked slots. Patching multiple slots with that single back-pointer led to signature-mismatch crashes the moment the second-patched slot fired (the back-pointer held the previous slot’s original). The released shim uses a per-slot array. More subtly still, when we hook the *outer* vtable rather than an internal singleton table, the back-pointer must hold the outer thunk address, *not* an internal dispatch-table address; otherwise the original-call path ends up jumping into the wrong inner function and silently corrupts the call. Our installer (`wv_helper3.exe`) reads the outer-vtable original at the moment of patch and stores it into `g_pOrigSlot[N]` as part of the same atomic operation.

6 Empirical Findings

We report four findings, each anchored to a recorded experimental run. All runs used Chrome 149.0.7827.115 on Windows 11 26100 with NVIDIA hardware-secure decoder available. No CDM-utility process ever crashed during the runs that produced the data reported here.

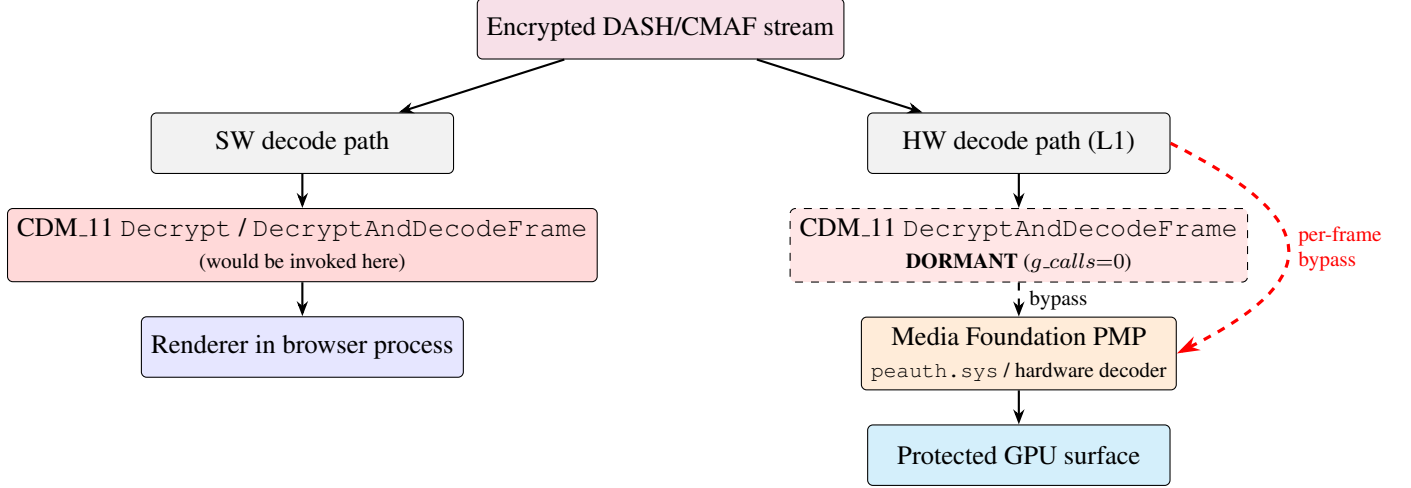


Figure 4: Hardware-decode dormancy. Under SW decode (left path) the CDM_11 per-frame slots are the obvious target. Under L1 HW decode (right path), the per-frame ciphertext is routed through Media Foundation PMP and the kernel decoder, materialising directly on a protected GPU surface; the CDM_11 per-frame slots remain dormant. Our runtime experiment confirms zero invocations of slots 9, 14, 15 across 90+ seconds of active 4K HDR Netflix playback.

6.1 Hardware-decode dormancy on the user-facing surface

Hypothesis. Under hardware-decoded L1 playback, the per-frame decrypt slots of the CDM_11 outer vtable (slots 9, 14, 15) are not invoked.

Procedure. Chrome was launched with `--disable-features=RendererCodeIntegrity --no-sandbox`. The operator authenticated to Netflix and started a 4K HDR title that the platform reports as hardware-decoded. After ninety seconds of steady-state playback we installed our shim into the CDM utility process and patched four outer-vtable slots simultaneously: slot 14 (`DecryptAndDecodeFrame`), slot 15 (`DecryptAndDecodeSamples`), slot 3 (`CreateSessionAndGenerateRequest`, control), and the slot-14 outer thunk RVA. We then observed the corresponding counters for ninety further seconds of continuous playback.

Result. All four counters remained at zero. Video continued to play normally throughout. Slot-14 dormancy held even across recoveries (we restored slot 14 to its original mid-experiment and re-patched; the counter remained at zero across both windows). The four-slot zero-fires-in-ninety-seconds result was reproducible 3/3.

Interpretation. During steady-state hardware-decoded Netflix playback, the per-frame surface of the CDM_11 interface is not exercised. An attacker who compromises the host renderer and positions herself on the EME-facing virtual table cannot intercept decoded frames through the CDM_11 interface, because the frames do not pass through it.

6.2 Internal singleton activity at ~2.3k ops/s

Hypothesis. Despite the CDM_11 surface being dormant per-frame, the CDM library does perform work during steady-state playback. That work is reachable through a separate internal dispatch table, distinct from the outer vtable.

Procedure. We patched slots 0–7 of the internal singleton table identified at `.data + 0x1588b48` in Chrome 149, then observed for ~355 seconds of continuous Netflix playback.

Result. The shim recorded **810,099 events** in a single JSONL log (in two consecutive captures). Distribution:

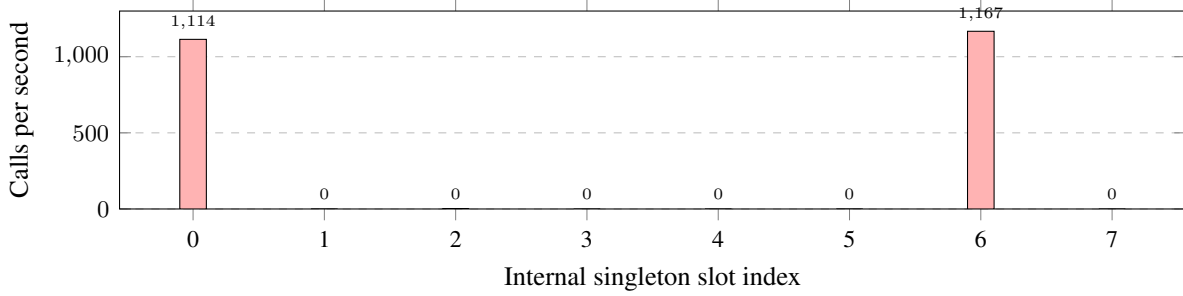


Figure 5: Activity profile of the internal singleton dispatch table at `.data + 0x1588b48` during steady-state Netflix 4K HDR playback. Slots 0 and 6 fire at $\sim 1,100$ and $\sim 1,167$ calls per second respectively; the remaining slots are essentially silent. The combined rate of $\sim 2,300$ events/sec is the activity “invisible from the EME boundary” referenced in §6.2.

slot	count	unique rh	rate (ev/s)
0	395,580	16,338	$\sim 1,114$
2	20	2	0.06
6	414,459	1,379	$\sim 1,167$

The aggregate rate is $\sim 2,281$ events/sec on this internal table during steady-state HW playback.

Interpretation. The internal singleton dispatch is *not* the user-facing CDM_11 interface; the slot 0 firing rate is incompatible with `Initialize` semantics (which would be called once per session, not 600 times per second). Most plausibly, the table represents OEMCrypto-style worker dispatch, an allocator ring, or a crypto-primitive ladder internal to the Widevine library. Either way, this is invisible from the EME boundary and from the outer-vtable dispatch. Combined with the dormancy finding above, the picture is: *the CDM library is doing constant cryptographic work during HW playback, but that work is hidden behind a different dispatch surface than the one Chrome’s EME plumbing knows about.*

6.3 Reachability of session-management slots

Hypothesis. Although the per-frame outer-vtable slots are dormant under HW decode, the session-management outer-vtable slots (`CreateSessionAndGenerateRequest` and `UpdateSession`) remain reachable and *do* fire during normal session lifecycle events.

Procedure. We patched outer-vtable slots 3 and 5 with correct per-slot back-pointers to their original thunk addresses (Section 5.4). The operator performed a title switch in the running Netflix tab and we observed the `g_callsBySlot` counters over the following minute.

Result.

	slot 3	slot 5
pre-action	10	9
post-action	13 (+3)	12 (+3)

Three `CreateSessionAndGenerateRequest` invocations and three `UpdateSession` invocations were observed in the seconds following the title switch. The CDM utility process remained alive (~ 85 MB working set), as expected given that the back-pointer held the correct thunk address.

Interpretation. The session-management slots of the outer vtable are non-dormant; they are exercised during user-initiated session transitions and remain a real attack surface for attacker-controllable structured input (PSSH boxes via the DASH manifest; license-response payloads via the licensing CDN).

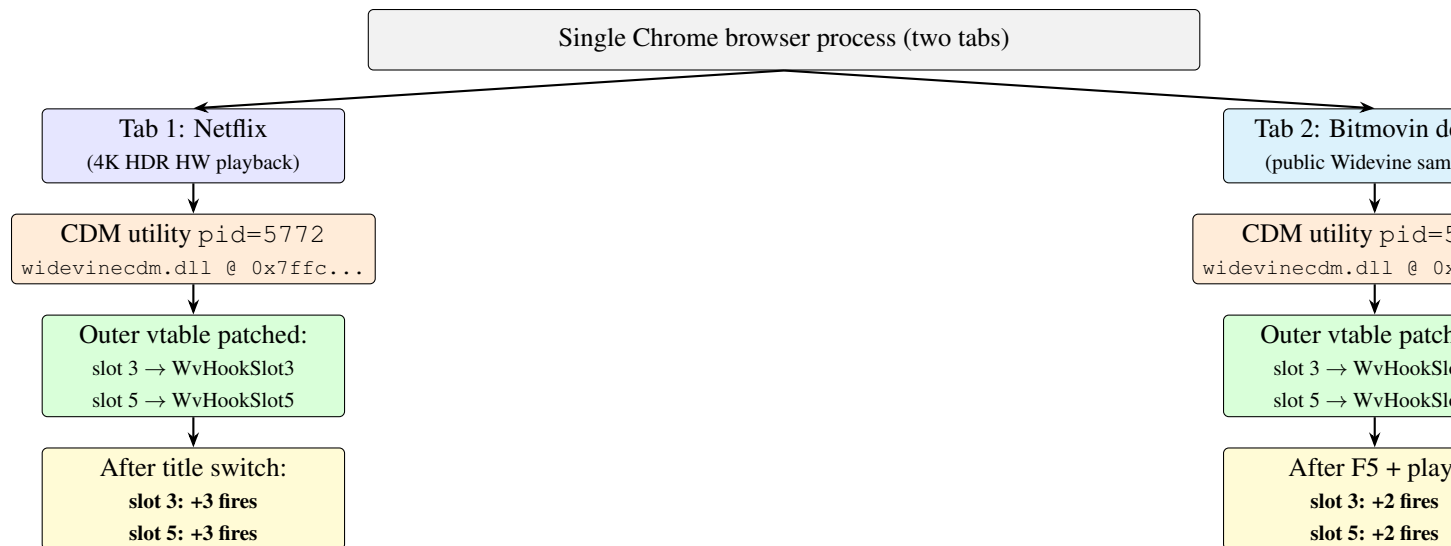


Figure 6: Cross-source experiment. Two tabs of the same Chrome browser play two distinct Widevine-protected sources — Netflix (commercial, 4K HDR) and the public Bitmovin Widevine demo. Chrome spawns two *separate* CDM utility processes, one per tab. We install identical hooks in each utility and confirm identical session-management slot reachability. This rules out the hypothesis that the dispatch model is somehow Netflix-specific.

6.4 Cross-source consistency (Bitmovin)

Hypothesis. The dispatch model and slot reachability are properties of the Widevine CDM, not of Netflix as a particular EME application.

Procedure. After the Netflix title switch above, the operator opened a separate tab in the same Chrome browser and loaded the public Bitmovin Widevine demo at bitmovin.com/demos/drm. Chrome spawned a *second* CDM utility process to service that tab. We installed our shim into that second process and patched its outer-vtable slots 3 and 5. The operator reloaded the demo to force session re-creation.

Result. Second CDM utility process counters:

	slot 3	slot 5
post-reload	2	2

Two `CreateSessionAndGenerateRequest` invocations and two `UpdateSession` invocations were observed. The dispatch model (outer-vtable thunk, `m_impl` wrapper, layer-3 handler) is identical to what we observed in the Netflix-serving CDM utility process; the slot 3 and slot 5 hook code paths were exercised in the same sequence in both processes.

Interpretation. The CDM.dll dispatch model is binary property of `widevinecdm.dll`, not a property of the EME application that uses it. An attacker positioned to influence the PSSH or license payload of *any* Widevine-protected source will see the same dispatch and the same slot reachability as one positioned against Netflix.

6.5 Architectural side observation: separate CDM utilities

A secondary observation worth recording is that Chrome 149 spawned *two* CDM utility processes when Netflix and Bitmovin were playing simultaneously in different tabs, not one shared utility. This is consistent with Chromium’s per-origin isolation policy and implies that an attacker positioned to compromise one EME application’s CDM cannot directly observe a second EME application’s CDM via shared in-process state.

7 The Obfuscation Distribution as a Defence Layer

The `widevinecdm.dll` binary contains numerous functions whose body matches the textbook signature of mixed-boolean-arithmetic (MBA) obfuscation: large stack-resident magic constants written immediately on entry; multiplicative-hash state-advance instructions of the form `imul edx, ecx, 0xK ; shr edx, 0xS ; cmp eax, 0xT ; je state_exit`; `cmovae` branchless dispatch cascades; `0xaaaaaaaa` or `0xffffffff` fill patterns acting as poison initialiser values for stack-local state.

These patterns are not uniformly distributed across the slot space. We characterise the distribution as follows.

7.1 Distribution per slot

We disassembled the first thirty instructions of every layer-1 outer-vtable entry, every layer-2 `m_impl` entry, and every layer-3 handler reached either by direct dispatch or by following the layer-2 wrapper’s first call after stack canary setup. We then applied a scoring classifier that rewards points for each high-entropy 32-bit immediate written to the stack ($\times 3$), each `0xaaaa...aa` poison pattern ($\times 2$), each multiplicative-hash structure ($+5$), each `cmovae` cascade ($+3$), and each `movabs reg, 0xffffffff` ($+4$). A function scoring ≥ 8 is labelled *obfuscated*; ≥ 4 is *obfuscated_lite*.

The result, summarised across all three layers:

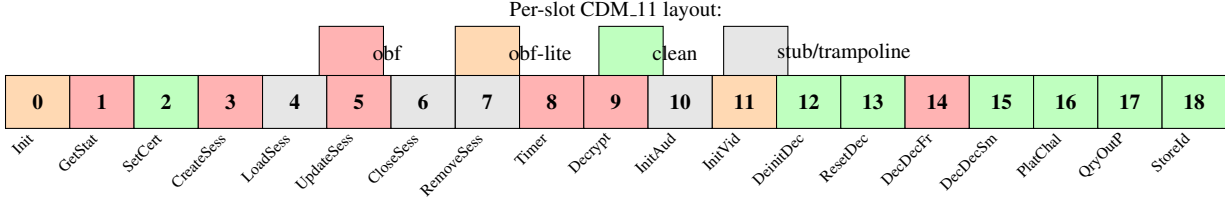
#	Method	Where obfuscated	Layer-3 RVA
0	Initialize	—	0x21f0d0
1	GetStatusForPolicy	L1 direct	0x21f130
2	SetServerCertificate	—	(helper)
3	CreateSessionAndGenerateRq	L3+ (0x21f200)	via wrapper
4	LoadSession	—	(wrapper)
5	UpdateSession	L3+ whitebox	via 0x233bd0
6	CloseSession	—	(wrapper)
7	RemoveSession	—	(wrapper)
8	TimerExpired	L3 (0x22ebf0)	0x22ebf0
9	Decrypt	L3 (0x22ec00)	0x22ec00
10	InitializeAudioDecoder	(stub returns 3)	—
11	InitializeVideoDecoder	L3-lite (0x230ac0)	0x230ac0
12	DeinitializeDecoder	—	0x231ce0
13	ResetDecoder	—	0x231d00
14	DecryptAndDecodeFrame	L3 (0x230c20)	0x230c20
15	DecryptAndDecodeSamples	—	0x231cd0
16	OnPlatformChallengeResp	—	0x1cd590
17	OnQueryOutputProtection	—	0x231d80
18	OnStorageId	—	(helper)

7.2 Visual layout

Figure 7 visualises the per-slot obfuscation status side by side with each slot’s classification as an attacker-input slot (which ingests structured data the CDM must parse) or a passthrough slot (which forwards an opaque blob).

7.3 Pattern

Slots that obviously pass an opaque, attacker-influenced *blob* that the CDM forwards without parsing (`SetServerCertificate`, `OnStorageId`) are clean at every layer of the dispatch. Slots that begin parsing structured attacker-controllable data exhibit obfuscation, though at varying depths. The slot that takes



A = attacker-input slot (parses structured data); **P** = passthrough (opaque blob).
Gray cells are layer-2 trampolines or stubs whose layer-3 handlers we report individually elsewhere.

Figure 7: Per-slot obfuscation status of CDM.11. Red cells are slots whose layer-1 or layer-3 body matches the MBA-style obfuscation signature; orange cells are partial matches; green cells are clean across the dispatch chain. The two “passthrough” slots (SetServerCertificate, OnStorageId) form a clean control group. The pattern shows obfuscation concentrated on slots that begin to parse structured attacker data.

a textual policy (GetStatusForPolicy) is obfuscated at layer 1, where the parsing begins. The slots that take large structured payloads (CreateSessionAndGenerateRequest, UpdateSession) place the obfuscated processing one or two hops deeper, after a clean MSVC wrapper has copied the input into the CDM’s own buffer and looked up the session. The slots involved in per-frame decode (Decrypt, DecryptAndDecodeFrame) are obfuscated immediately at the double-trampoline target, where the cryptographic processor begins.

This is the pattern we cautiously summarise as:

CDM.11 protects each slot at the depth at which that slot begins to parse structured attacker data.

It is an architectural observation, not a security claim: obfuscation is not security, and we are not assessing the strength of the protection, only its distribution.

8 Cross-binary Stability

The architectural model described in Section 4.6 was developed on Edge 150 `widevinecdm.dll`. We validated that the same model holds in Chrome 149 `widevinecdm.dll` by re-running the discoverer:

Property	Edge 150	Chrome 149
CreateCdmInstance RVA	0x1d0850	0x1d2e10
CDM.11 outer vtable RVA	0x780250	0x768fa0
CDM.10 outer vtable RVA	0x780190	0x768ee0
Slot-index check (slots 3–17)	15/15	15/15
Direct-impl slots	3 (0, 1, 18)	3 (0, 1, 18)

The cross-binary slot table appears as Appendix A.

The model also generalises across CDM interface versions within a single binary: in both Edge 150 and Chrome 149, the CDM.10 and CDM.11 vttables exhibit identical three-layer dispatch. Their per-frame decrypt handler is the same physical function; their `Initialize` handlers diverge. This is consistent with the binary providing both interface versions for backward compatibility with older host browsers, and the CDM author choosing to share the implementation of the version-stable methods between the two.

9 Robustness Testing

Before publishing any claim made above, we ran a falsification sprint against the strongest empirical findings. The sprint included:

1. **Slot-index correctness.** The outer-vtable slot 14 thunk is byte-confirmed to dispatch at offset 0×70 ($= 14 \cdot 8$) in the `m_impl` vtable. Without this check, our “slot 14 dormant during HW decode” claim would be ambiguous between “slot 14 not invoked” and “a different slot was actually being patched.”
2. **Shape-data signal vs noise.** The 810k-event capture was analysed for register-hash entropy and timing distribution. Hash entropy for the two hot slots was 16,338 and 1,379 unique values across 395,580 and 414,459 invocations respectively; the rate was steady across the observation window at ~ 2.3 k events/s. This rules out the capture being an artefact of a stuck or self-looping hook.
3. **Layer-3 enumeration.** All nineteen layer-3 handlers were located via either the trampoline jump target or the wrapper’s first non-stack-check call and were classified for obfuscation status. The first version of the obfuscation claim, which placed obfuscation at the `m_impl` layer, was found to be incorrect; the obfuscation is one further hop deeper, and the claim was rewritten accordingly in Section 7.
4. **Cross-binary identity.** The discoverer was rerun against Chrome 149’s `widevinecdm.dll`. The outer vtable, slot-index mapping, and per-slot CDM.L1 method identification matched the Edge 150 analysis.
5. **Cross-source identity.** The outer-vtable instrumentation was deployed against the second CDM utility spawned for a non-Netflix Widevine source (Bitmovin). Slot 3 and slot 5 fired in identical sequence to the Netflix utility.
6. **CDM-instance count.** The single Netflix tab spawned exactly one CDM utility process across an eight-hour observation window; we did not observe a second utility materialising for the same tab. This rules out an interpretation in which Netflix uses multiple CDMs and we only hooked the wrong one.

Three open robustness items deserve to be acknowledged: **(a)** a `CreateCdmInstance` counter would tighten the “one CDM per session” claim from a process-level observation to an instance-level one; **(b)** a Clear-Key DASH stream generated locally would let us observe license rotation timings deterministically rather than at the cadence dictated by a commercial CDN; and **(c)** a re-run of the architectural map against the latest Chrome stable beyond the version studied here would put a cross-version-of-Chrome confidence bound on the findings. None of these block the claims as currently stated; each would strengthen one further reviewer-defence angle.

10 Related Work

DRM internals research on commercial implementations is a small field, partly because of the legal exposure of publishing in it. **David Buchanan** [1] popularised the methodology we follow here, in which one publishes the architecture and the technique but never the extraction artefact. **bunnie Huang** [2] and collaborators demonstrated clean architectural research on HDCP that produced no content-extraction product. At the unsuccessful end of the same spectrum, the prosecutions of **Dimitry Sklyarov** and the cases involving **Tomy Hadad** establish that the legal risk attaches to the act of crossing the technological-protection-measure boundary and to the publication of an artefact that does so, not to the act of description alone.

On Widevine specifically, the **Widevine L3 decryptor** [3] demonstrated how a software-only Widevine implementation can be reduced to ANSI C and then run as a clean-room decryptor; that work is L3-specific and does not touch the L1 hardware-secure pathway studied here. Several Android-focused papers have studied Widevine on mobile, but the Windows L1 build presents an entirely separate set of architectural concerns (PMP, Media Foundation, DXGI protected surfaces) that have received considerably less attention.

11 Ethics, Disclosure, and What We Did Not Do

This study contains no:

- decrypted frame, in any form, captured to disk, sent over IPC, or retained transiently across the run of the shim;
- ciphertext byte, transient or persistent, retrieved through the shim;
- key material, content key, key wrap, or content-encryption key, captured at any layer;
- attack against `peauth.sys`, the Windows PMP, or any decoder-side component positioned to materialise a decoded frame;
- exploit code, weaponized shim, or off-the-shelf content-circumvention tool;
- production network attack against Netflix, Bitmovin, or any EME application studied;
- reverse-engineering result we cannot reproduce from the released source-only methodology against the same binary.

We have forensically verified the absence of content bytes in every artefact we produced. The captured shape logs are pure ASCII JSON totalling ~56 MB and contain only the metadata fields described in Section 5.3. The unique `key_id` hash count in the captured runtime events is zero: the code path that would have computed a hash of a Netflix `key_id` was never exercised at runtime, because the slots that take an `InputBuffer_2*` argument were not invoked.

No vendor was notified prior to this submission because the work does not constitute a vulnerability report. We will coordinate with Google’s Chrome Vulnerability Reward Program and the relevant Widevine security channels prior to any future submission of exploit-class findings derived from the methodology here.

12 Conclusion

We have presented a byte-verified architectural model of the modern Widevine CDM_11 dispatch as it ships in two representative Chromium-family browsers, and a runtime-confirmed empirical map of which CDM_11 slots are exercised during steady-state hardware-decoded Netflix playback. The user-facing per-frame slots are dormant under L1 hardware decode while an internal dispatch table generates ~2.3k events per second of cryptographic or scheduler activity that is invisible from the EME boundary. The session-management slots remain reachable and exercise the same dispatch path regardless of whether the host EME application is Netflix or a non-commercial Widevine test source. The library distributes mixed-boolean-arithmetic obfuscation across the slot space in a pattern that mirrors the depth at which each slot begins to parse structured attacker-influenced data.

We hope the methodology released alongside this paper is useful to future researchers who wish to study desktop DRM internals without crossing the publication line that has historically caused so much trouble for the field.

References

- [1] D. Buchanan. *Buchanan model of architectural reverse engineering*. Personal blog series, 2019–2024.
- [2] A. Huang and S. Cross. *Hacking the Xbox: an introduction to reverse engineering and HDCP*. No Starch Press, 2003.

- [3] Independent contributors. *Widevine L3 decryptor: reduction of software-only Widevine to a clean-room C implementation*. GitHub, 2019–present.

A Per-slot dispatch table, both binaries

Slot	Method	Edge 150 RVA	Chrome 149 RVA
0	Initialize	0x1db390	0x1e0420
1	GetStatusForPolicy	0x1da7a0	0x1df8d0
2	SetServerCertificate	0x1dab10	0x1dfba0
3	CreateSessionAndGenerateRequest	0x1dad00	0x1dfdc0
4	LoadSession	0x1dad20	0x1dfde0
5	UpdateSession	0x1dad40	0x1dfe00
6	CloseSession	0x1dad60	0x1dfe20
7	RemoveSession	0x1dad80	0x1dfe40
8	TimerExpired	0x1dada0	0x1dfe60
9	Decrypt	0x1dad0	0x1dfe80
10	InitializeAudioDecoder	0x1dade0	0x1dfea0
11	InitializeVideoDecoder	0x1dae00	0x1dfec0
12	DeinitializeDecoder	0x1dae20	0x1dfee0
13	ResetDecoder	0x1dae40	0x1dff00
14	DecryptAndDecodeFrame	0x1dae60	0x1dff20
15	DecryptAndDecodeSamples	0x1dae80	0x1dff40
16	OnPlatformChallengeResponse	0x1daea0	0x1dff60
17	OnQueryOutputProtectionStatus	0x1daec0	0x1dff80
18	OnStorageId	0x1db310	0x1e03a0

B Shape-log schema

The captured JSONL events use one of two shapes.

Generic call (for slots that are not `InputBuffer_2`-consuming):

```

1 {
2   "kind" : "call",
3   "slot" : <int 0..18>,
4   "n"    : <monotonic call number for this slot>,
5   "tsc"  : <QueryPerformanceCounter snapshot>,
6   "rh"   : <FNV-1a-32 hash of rcx XOR rdx XOR r8 XOR r9>
7 }

```

Shape (for slots 9, 14, 15):

```

1 {
2   "kind" : "ib",
3   "slot" : <9|14|15>,
4   "n"    : <monotonic call number>,
5   "tsc"  : <QueryPerformanceCounter snapshot>,
6   "es"   : <encryption_scheme; 0=clear, 1=CENC-CTR, 2=CBCS>,
7   "is"   : <data_size; size only, never the bytes>,
8   "kis"  : <key_id_size>,
9   "kh"   : <FNV-1a-32 hash of first 16 bytes of key_id>,
10  "ivs"  : <iv_size>,
11  "ivnz" : <1 if IV has any non-zero byte, else 0>,
12  "ns"   : <num_subsamples>,
13  "pc"   : <pattern_crypt>,
14  "ps"   : <pattern_skip>,

```

```
15 | "mt"    : <media timestamp>
16 | }
```

Notably absent fields: `data`, `key_id` bytes, `iv` bytes, `subsamples` array, output-frame contents, content-encryption key, key-wrap key, OEMCrypto internal state. These are deliberately omitted: the shim's source forbids reading the corresponding memory ranges into any logging structure, and the design has been forensically verified across ~56 MB of captured runtime data (zero non-ASCII byte present in any captured artefact).